



Sistem üzerinden yayımlanan içerikler Sabancı Üniversitesi'nin bünyesinde kullanılmak ve dağıtılmak üzere oluşturulmuştur. İçeriklerin fikri mülkiyet hakları herhangi bir süre sınırlı olmaksızın, Sabancı Üniversitesi'ne aittir. Sabancı Üniversitesi'nin önceden yazılı izni alınmaksızın üçüncü şahıslara kullanılamaz, kopyalanamaz ve çoğaltılamaz.

The contents published on the system were created to be used and distributed within Sabancı University. The intellectual property rights of the contents belong to Sabancı University without any limitation. It cannot be used, copied or reproduced to third parties without the prior written permission of Sabancı University.



CS310

Mobile Application Development

Flutter - Firebase



Outline

- Firebase Overview
- Firebase Settings
- Firebase
 - Core Package
 - Authentication Service
 - Firestore Database
 - Analytics Service
- Stream



Firestore

- Firestore is a Backend-as-a-Service (BaaS) platform by Google that provides tools and services for building, managing, and scaling mobile and web apps.
- It offers features like real-time databases, cloud storage, authentication (email, social logins), push notifications, analytics, and hosting.
- Firestore simplifies backend development by handling infrastructure, allowing developers to focus on the front end.



Why Firebase in Mobile Apps?

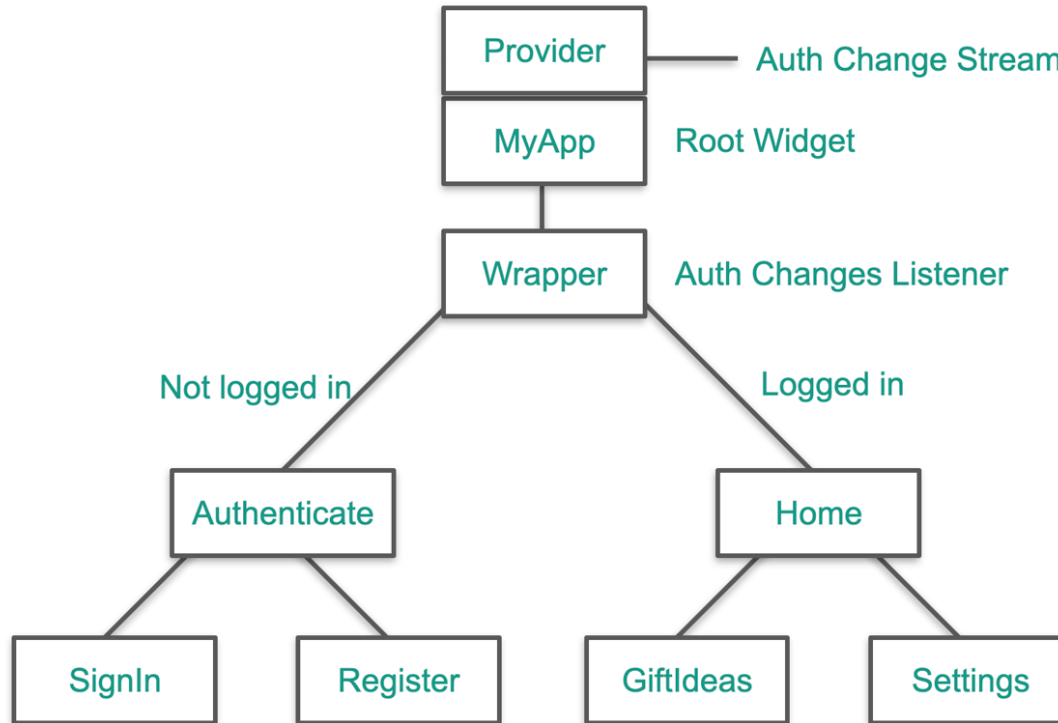
- No servers required
- Scales automatically
- Real-time updates
- Easy Flutter SDK
- Secure (with Firebase Rules)
- Great for student projects & rapid prototyping



Firestore

- Firestore is free for low-usage scenarios within the Spark Plan's quotas, but it's not "unlimited" free. If your app scales (e.g., thousands of users, heavy database queries, or large file downloads), you'll likely hit limits and incur costs on the Blaze Plan.
- To stay within the free tier, optimize database design (e.g., minimize reads/writes), reduce image sizes, and monitor usage via the Firestore console.

Sample Project Gift Idea - Widget Tree

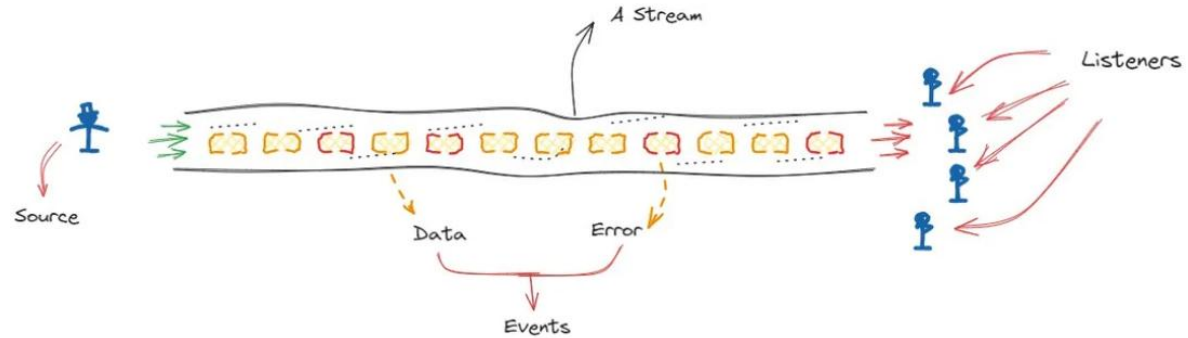




Understanding Streams in Dart & Flutter (Before Firebase)

- What is a Stream?
- Why Firebase depends on Streams
- How to use StreamBuilder

Stream



- A Stream is a **sequence of asynchronous data (events) delivered over time**, used for real-time updates in Flutter apps.
- Examples:
 - Firebase Authentication's stream provides real-time updates to manage user sessions in a Flutter app
 - Listening to a Firestore query for live data changes or handling user input in real-time.
- Key Classes: Stream (data source), StreamBuilder (UI updates based on stream data).



Why do we need Streams?

Because many things in apps happen **continuously over time**:

- Firebase real-time updates
- Auth state changes
- Sensor data
- TextField typing events
- Timers
- Network real-time feeds

So we need a mechanism that gives **multiple async values**, not just one.



Future vs Stream

Feature

Future

Stream

Returns

ONE value

MANY values

Timing

async once

async repeatedly

Example

login()

Firebase snapshots

UI widget

FutureBuilder

StreamBuilder



Types of Streams

1. **Single-subscription streams** → common for Firebase
2. **Broadcast streams** → multiple listeners



Creating and Listening a Simple Stream

```
Stream<int> numberStream() async* {  
    for (int i = 1; i <= 5; i++) {  
        await Future.delayed(Duration(seconds: 1));  
        yield i;  
    }  
}  
  
void main() {  
    numberStream().listen((value) {  
        print(value);  
    });  
}
```



StreamBuilder Widget (Most Important for Firebase)

```
StreamBuilder<int>(  
  stream: numberStream(),  
  builder: (context, snapshot) {  
    if (!snapshot.hasData) return CircularProgressIndicator();  
    return Text("Value: ${snapshot.data}");  
  },  
)
```



Step by Step Firebase Project



Step 1: Create Firebase Project

- Go to <https://console.firebase.google.com>
- Click Add Project
- Enter project name
- Disable Google Analytics for now



Step 2: Add Android App

Add new app → Android

Enter:

- Android package name
- App nickname

Download google-services.json

Put file in:

`android/app/google-services.json`



Step 3: Add iOS App

- Add iOS bundle ID
- Download **GoogleService-Info.plist**
- Add to Runner project



Step 4: Update Android build files

```
android/build.gradle  
classpath 'com.google.gms:google-services:4.3.15'
```

```
android/app/build.gradle  
apply plugin: 'com.google.gms.google-services'
```



Step 5: Add Firebase packages in pubspec

```
firebase_core: ^3.0.0  
firebase_auth: ^5.0.0  
cloud_firestore: ^5.0.0  
firebase_storage: ^12.0.0
```

And Run:

```
flutter pub get
```



Step 6: Initialize Firebase in main.dart

```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await Firebase.initializeApp();  
  runApp(MyApp());  
}
```



Firestore – Core Package

- The Firestore Core package (often referred to as **firebase_core** in SDKs like Flutter) is the foundational library required to initialize and configure Firestore services in an app.
- It enables the app to **connect to the Firestore backend**, set up project-specific settings, and integrate other Firestore services like Authentication, Firestore, or Analytics.
- **Without it, other Firestore features won't work.**



Firebase – Core Package – InitializeApp

```
...
// Import the firebase_core plugin
import 'package:firebase_core/firebase_core.dart';
...
@override
Widget build(BuildContext context) {
  return FutureBuilder(
    // Initialize FlutterFire
    future: Firestore.initializeApp(),
    builder: (context, snapshot) {
      // Check for errors
      if (snapshot.hasError) {
        return MaterialApp(
          home: FlutterFire.initializeApp());
      }
      // Once complete, show your application
      if (snapshot.connectionState == ConnectionState.done) {
        return MaterialApp(
          home: FlutterFire.initializeApp());
      }
    }
  );
}
```



FlutterFire

FlutterFire is the official set of Flutter plugins that lets your Flutter app connect to Google Firebase services easily.

- Flutter = Mobile app (UI + logic)
- Firebase = Backend services (auth, database, storage, etc.)
- FlutterFire = The bridge that connects Flutter with Firebase



What we Can Do with FlutterFire

With FlutterFire, we can easily add:

Authentication → Login with Email, Google, Apple, etc.

Cloud Firestore → Real-time NoSQL database

Firebase Storage → Upload images, files, videos

Push Notifications → Firebase Cloud Messaging (FCM)

Analytics & Crash Reports

Hosting & Remote Config



Firestore – Authentication Service

- Firestore Authentication is a service that simplifies **user authentication** for apps.
- It supports email, password, social media logins (like Google, Facebook), and anonymous sign-ins, handling secure user management, token generation, and integration with other Firestore services.



Firestore – Firestore Database Service

- Firestore is a **NoSQL cloud database** for storing and syncing data in real-time.
- It supports scalable, hierarchical data structures, offers offline capabilities, and integrates seamlessly with Firebase Authentication and other Firebase services for building responsive apps.
 - **Collection:** A group of documents, similar to a folder or table. It organizes data and contains multiple documents but doesn't store data itself.
 - **Document:** An individual record within a collection, akin to a row in a table. It stores data as key-value pairs (JSON-like) and can contain nested objects or references to subcollections.



Firebase – Analytics Service

- Firebase Analytics is a free tool that **tracks user behavior and app usage**, providing insights through event-based data collection.
- It logs predefined or custom events (e.g., app opens, purchases), offers audience segmentation, and integrates with other Firebase services for targeted campaigns and performance analysis.



What Is a Firebase App ID?

A **Firebase App ID** is like a **unique identity card** for each version of your app:

- Android app → gets its own ID
- iOS app → gets its own ID
- Web app → gets its own ID

Firebase uses these IDs to:

- Track analytics
- Apply authentication rules
- Manage Firestore access
- Handle notifications correctly per platform



Flutter Bottom Sheet

- A Flutter Bottom Sheet is a UI component that slides up from the bottom of the screen to display additional content or options.
- It's typically used for menus, forms, or confirmations.
 - Persistent: Stays visible, part of the app's layout (e.g., BottomSheet).
 - Modal: Temporary, dismissible overlay (e.g., showModalBottomSheet).

A UI Container for Authentication Forms

```
showModalBottomSheet( context: context,  
  builder: (context) => Container( height: 200,  
                                   child: Text('Bottom Sheet Content'), ), );
```

Firebase Codelab Project

The app uses the following Firebase products:

Authentication: Lets users sign in to your app.

Firestore: Saves structured data on the cloud and gets instant notifications when data changes.

Firebase Security Rules: Secures your database.

Codelab Project Video: <https://youtu.be/wUSkeTaBonA>

Follow along with the codelab: <https://goo.gle/3vTnH08>

Introduction and Prerequisites

Running the sample project

Creating a Firebase project

Configuring the iOS and Android versions

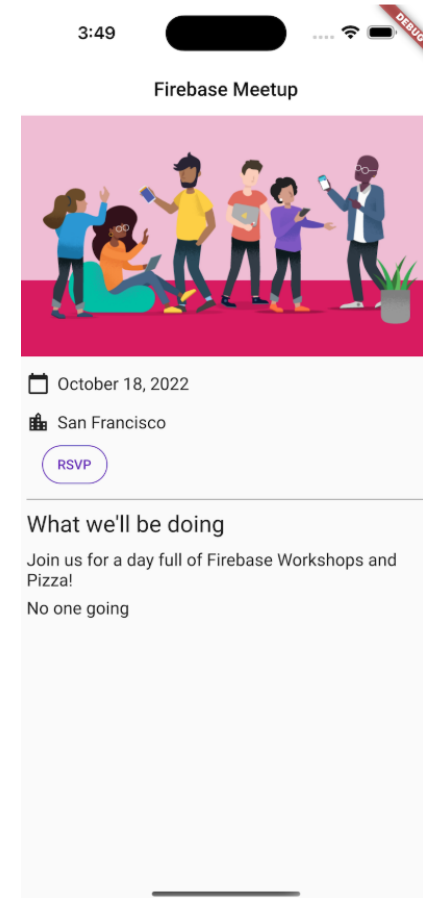
Enabling user sign-in with Firebase Auth

Writing documents to Firestore

Reading documents from Firestore in real-time

Adding security rules

Bonus step: Adding RSVP counts





Firestore Codelab Project

The Key Takeaways



What Is `firebase_options.dart`?

This file is auto-generated by Firebase CLI.

It contains:

- Your Firebase Project ID
- App ID
- API keys (safe version)
- Platform configs (Android, iOS, Web, etc.)



Why Did They Add go_router?

Before, we have used navigation like this:

Navigator.push(...)

That works for **small apps**.

But Firebase apps need:

- Login page
- Register page
- Home page
- Profile page
- Protected routes

So they use **go_router** instead.



What Is `go_router`

`go_router` is a clean and powerful navigation system for Flutter.

It decides:

Which screen to show, based on:

- Login status
- URL
- Route name



How go_router Works

//Instead of navigation.push()..

```
final GoRouter router = GoRouter(  
  routes: [  
    GoRoute(  
      path: '/',  
      builder: (context, state) => HomeScreen(),  
    ),  
    GoRoute(  
      path: '/login',  
      builder: (context, state) => LoginScreen(),  
    ),  
  ],  
);
```



How go_router Works-cont

//Then in Material App

```
MaterialApp.router(  
  routerConfig: router,  
);
```



Why go_router Is Used With Firebase?

Because Firebase has:

1. Logged-in users
2. Logged-out users

So router can do:

```
if (user is logged in)
  go to Home
else
  go to Login
```

This is impossible to manage cleanly using only Navigator.push.



Authentication.dart

//First Button (RSVP / Logout)

```
StyledButton(  
  onPressed: () {  
    !loggedIn ? context.push('/sign-in') : signOut();  
  },  
  child: !loggedIn ? const Text('RSVP') : const Text('Logout'),  
)
```



Authentication.dart

//Second Button → Profile (Only When Logged In)

Visibility(
 visible: loggedIn,

It means : “Only show this widget when loggedIn == true.”



Authentication.dart

//Second Button → Profile (Only When Logged In)

```
StyledButton(  
  onPressed: () {  
    context.push('/profile');  
  },  
  child: const Text('Profile'),  
),
```



Homepage.dart

HomePage now listens to Firebase login state using Provider and shows Login/Logout/Profile buttons dynamically using the AuthFunc widget.

```
Consumer<ApplicationState>(
  builder: (context, appState, _) => AuthFunc(
    loggedIn: appState.loggedIn,
    signOut: () {
      FirebaseAuth.instance.signOut();
    },
  ),
),
```



What is This Sign-Out Function? (home.darto

```
signOut: () {  
  FirebaseAuth.instance.signOut();  
}
```

This means: "When Logout is clicked : tell Firebase to log the user out." This triggers:

- Firebase logout
 - Firebase notifies ApplicationState
 - loggedIn becomes false
- UI updates automatically



Data Model

- Firestore is a NoSQL database (Not only SQL), and data stored in the database is split into collections, documents, fields, and subcollections.
- In this codelab, a guestbook data model has been used using firestore
- A guestbook is a public message wall where logged-in users can leave short messages that everyone can see.
- You store each message of the chat as a document in a guestbook collection, which is a top-level collection.



Resources

- Firebase Overview: <https://firebase.google.com/docs/projects/learn-more>
- Firebase Console: <https://console.firebase.google.com/>
- Firebase Settings :
<https://firebase.google.com/docs/flutter/setup?platform=android>
- All available plugins:
<https://firebase.google.com/docs/flutter/setup?platform=android#available-plugins>
- For required packages: <https://pub.dev/>